Smart Change Calculator:
Greedy vs. Dynamic Programming

CSCE 4110 - Algorithms

Final Project Report

Project member: Bryson Brown

Date: November 22, 2026

Abstract / Executive Summary

This project compares a largest-denomination-first greedy method with bottom-up dynamic programming for the minimum coin-change problem. A C++17 application accepts change amounts and coin denominations, reconstructs selected coins, and measures execution time and operation counts. Representative cases demonstrate that greedy can be suboptimal or unable to find an existing solution. A local MacBook benchmark with 30 trials for each of six amounts shows dynamic-programming runtime increasing with target amount while greedy remains relatively low for the tested three-coin set.

1. Introduction

The coin-change problem asks for the smallest number of coins that sum to a requested amount. The project illustrates the tradeoff between a fast heuristic and an exact optimization method. Goals include correct implementation, understandable outputs, meaningful edge-case testing, and empirical performance comparison.

2. Background

Greedy repeatedly chooses the largest denomination not exceeding the remaining amount. It is not guaranteed to find an optimal solution for arbitrary coin systems. Bottom-up dynamic programming solves smaller amounts first and uses those solutions to construct a minimum-coin answer when one exists.

3. System / Algorithm Design

Greedy: sort unique positive denominations from largest to smallest; take as many of each as possible; report success only when the remainder is zero. Dynamic programming: initialize dp[0] = 0 and other entries as unreachable; for each amount a, consider each coin c where a - c is reachable, minimizing dp[a-c] + 1. Save predecessor choices to reconstruct the selected coins.

For target amount A and k denominations, the bottom-up method requires O(Ak) time and O(A) auxiliary space. Greedy requires O(k log k) sorting and O(k) denomination visits, plus

memory/time to materialize selected coins. These bounds describe different guarantees: greedy is a heuristic, while DP finds a minimum-coin solution.

4. Implementation

The program uses C++17 and standard library components including vector, algorithm, chrono, iostream, and fstream. The main application is in Smart_Change_Calculator/src/main.cpp. The benchmark program in Phase3/benchmark.cpp calls the existing greedy and dynamicProgramming functions, runs repeated trials, and writes benchmarks.csv. The current benchmark imports main.cpp by temporarily renaming main; separating reusable functions into their own module would improve maintainability.

5. Testing and Validation

Amount	Denominations	Greedy	DP
30	1, 10, 25	6 coins	3 coins
6	3, 4	No solution found	2 coins
3	2, 4	Impossible	Impossible
87	1, 5, 10, 25	6 coins	6 coins

Table 1. Representative expected correctness cases from the project testing plan.

These cases test greedy suboptimality, a greedy dead end, unreachable change, and an ordinary coin system. The earlier automated tests were reported as passing; the team should rerun and retain their output before final submission. Benchmark success by itself does not verify all correctness cases.

6. Results and Analysis

The following results were transcribed from the successful VS Code Terminal benchmark run on Bryson Brown's MacBook. The harness used denominations {1, 10, 25}, target amounts from 100 to 10,000 cents, and 30 trials per amount. Values are mean microseconds as printed by the program; these timings are machine-specific and should not be treated as universal performance guarantees.

Amount (cents)	Greedy mean (µs)	DP mean (µs)
100	0.110	0.610
500	0.281	1.893
1000	0.264	3.439
2500	0.333	8.126
5000	0.377	15.414

10000	0.518	30.254

Table 2. Local MacBook benchmark averages, 30 trials per amount.

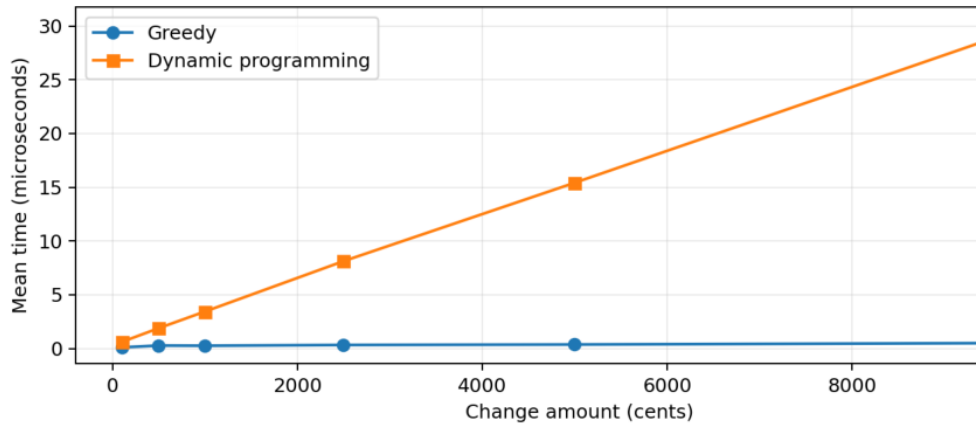


Figure 1. Measured average execution time versus requested change amount.

Figure 1. Measured average execution time versus requested change amount.

At 10,000 cents, greedy averaged 0.518 µs versus 30.254 µs for dynamic programming (about 58 times as long). DP time rises roughly with amount, consistent with O(Ak) for fixed k. Greedy time is small and less sensitive to amount for these denominations, although reconstructing many selected coins can increase its work. At sub-microsecond durations, measurement noise, clock overhead, and compiler optimizations can materially affect results. The reported operation counters also measure different actions and should not be interpreted as directly equivalent units.

7. Demonstration Summary

The planned video will show compilation, a normal change request, the 30-cent greedy counterexample, an unreachable case, and the benchmark run generating benchmarks.csv. The final public video URL and reproducible repository URL must be inserted after upload and tested in a private browser window.

8. Team Contributions

Bryson Brown — sole intended project member: responsible for integrating and running the C++ application, executing the local benchmarks, reviewing the report, and presenting the demonstration. Before final submission, update this statement to reflect the actual implementation, testing, report-writing, and video work completed. Approval needed: the original project guidelines specify 2–3 students; solo submission requires instructor confirmation.

9. Challenges and Lessons Learned

An initial benchmark build failed because its source include referenced a temporary generation directory. Correcting the relative path between Phase3 and Smart_Change_Calculator resolved the compilation issue. The CSV destination was also changed to a relative path so results are saved in the active working directory. This highlighted the importance of portable paths, reproducible builds, and running code in the actual submission environment.

10. Conclusion

The project demonstrates that greedy coin selection can be computationally inexpensive but unreliable for arbitrary denominations, while dynamic programming provides an optimal answer at a higher cost dependent on the requested amount. The local benchmark confirms the expected broad scaling trend for the tested coin set. Future work could add independent algorithm modules, more randomized correctness tests, and benchmark distributions with confidence intervals.

References

CSCE 4110, Semester Project Guidelines and Final Project Submission Guidelines (course-provided documents).

C++ standard library reference, cppreference.com (vector, algorithm, chrono, fstream).

Project C++ source code and locally generated benchmark output; GitHub URL: https://github.com/brysonb24/CSCE4110-Smart-Change-Calculator.